Deep RAPM

1 Deep Learning Extensions of Regularized Adjusted Plus Minus (RAPM)

In this section, we propose several neural architectures that extend the classical Regularized Adjusted Plus Minus (RAPM) framework by learning latent offensive and defensive player representations. Recall that in standard RAPM, we predict a possession?level outcome (e.g., points scored, an ordinal rating, or a binary indicator) as a linear combination of ?offensive? and ?defensive? indicator variables?leading to a design matrix with $2N_{\text{players}}$ columns (one column for each player in an offensive role, one for each in a defensive role). Formally, for possession i, one writes

$$\hat{y}_i \;=\; f_\theta\big(\mathbf{o}_i,\, \mathbf{x}_i,\, \mathbf{g}_i\big),$$

where:

- $\mathbf{o}_i = \{\, o_{i,1}, \ldots, o_{i,5}\}$ are the indices of the five **offensive** players on the court during possession i,
- $\mathbf{x}_i = \{\, x_{i,1}, \ldots, x_{i,5}\}$ are the indices of the five **defensive** players,
- $\mathbf{g}_i \in \mathbb{R}^p$ is a vector of **gamestate features** (e.g., score differential, time remaining, home/away indicator),
- θ are the learnable parameters of f_θ (in classical RAPM, this is simply a vector of coefficients plus a ridge penalty).

Rather than assigning each player a single scalar coefficient on offense and defense, we instead learn **two latent embeddings** per player:

$$u_j^o \in \mathbb{R}^d, \quad u_j^d \in \mathbb{R}^d, \qquad j = 1, \ldots, N_{\text{players}},$$

where u_j^o is the *offensive embedding* of player j, and u_j^d is the *defensive embedding* of player j. We then combine the five offensive embeddings into a single *offensive lineup embedding* h^o, and similarly combine the five defensive embeddings into h^d. Finally, we fuse $(h^o,\, h^d,\, \mathbf{g})$ to predict the outcome. Below, $\text{SetEncoder}_\theta(\cdot)$ denotes a permutation?invariant aggregator (e.g., Deep Sets or a Set Transformer). Our generic prediction rule becomes

$$h_i^o \;=\; \text{SetEncoder}_\theta\big(\{\, u^o_{o_{i,j}}\}_{j=1}^5\big), \quad h_i^d \;=\; \text{SetEncoder}_\theta\big(\{\, u^d_{x_{i,k}}\}_{k=1}^5\big), \tag{1}$$

$$\hat{y}_i \;=\; f_\theta\big(h_i^o,\, h_i^d,\, \mathbf{g}_i\big). \tag{2}$$

Below we detail one such architecture?**DeepRAPM-Attn**?which uses a Set Transformer to handle permutation invariance and capture intra?lineup interactions.

1.1 DeepRAPM-Attn (Set Transformer / Deep Sets)

Overview. We build two separate ?set encoders? for the offensive and defensive lineups, each of which learns to aggregate five player embeddings $\{u_j^o\}$ or $\{u_k^d\}$ into a single vector in $\mathbb{R}^d$. Concretely, let

$$\big\{\, u^o_{o_{i,1}},\, u^o_{o_{i,2}},\, u^o_{o_{i,3}},\, u^o_{o_{i,4}},\, u^o_{o_{i,5}}\big\} \quad \subset \mathbb{R}^d, \quad \big\{\, u^d_{x_{i,1}},\, u^d_{x_{i,2}},\, u^d_{x_{i,3}},\, u^d_{x_{i,4}},\, u^d_{x_{i,5}}\big\} \quad \subset \mathbb{R}^d,$$

denote the *offensive* and *defensive* player embeddings on possession i. We feed each set into a shared, parameterized *set encoder* (either Deep Sets or a Set Transformer), obtaining

$$h_i^o \;=\; \mathrm{SetEncoder}_\theta\Big(\{\, u^o_{o_{i,j}}\}_{j=1}^5\Big) \;\in\; \mathbb{R}^d, \quad h_i^d \;=\; \mathrm{SetEncoder}_\theta\Big(\{\, u^d_{x_{i,k}}\}_{k=1}^5\Big) \;\in\; \mathbb{R}^d. \tag{3}$$

We then concatenate these lineup embeddings with the gamestate vector $\mathbf{g}_i \in \mathbb{R}^p$, and pass the result into a prediction MLP:

$$z_i \;=\; \big[\, h_i^o;\; h_i^d;\; \mathbf{g}_i \,\big] \;\in\; \mathbb{R}^{(2d+p)}, \quad \hat{y}_i \;=\; \mathrm{MLP}_\theta\big(z_i\big) \;\in\; \mathbb{R}. \tag{4}$$

Below, we describe two alternatives for $\mathrm{SetEncoder}_\theta$, and give the full attention?based equations when using a Set Transformer.

1.1.1 (A) Deep Sets Encoder

A *Deep Sets* encoder zaheer2017deep takes an unordered set $\{\, u_1, \ldots, u_m\}$, applies a shared MLP $\phi_\theta : \mathbb{R}^d \to \mathbb{R}^r$ to each element, sums (or takes mean) the outputs, and then applies another MLP $\rho_\theta : \mathbb{R}^r \to \mathbb{R}^d$. Concretely:

$$\phi_\theta\big(u^o_{o_{i,j}}\big) \;\in\; \mathbb{R}^r, \quad j = 1 \ldots 5, \tag{5}$$

$$s_i^o \;=\; \frac{1}{5}\sum_{j=1}^{5} \phi_\theta\big(u^o_{o_{i,j}}\big) \;\in\; \mathbb{R}^r, \quad h_i^o \;=\; \rho_\theta\big(s_i^o\big) \;\in\; \mathbb{R}^d, \tag{6}$$

$$\text{(similarly) } h_i^d \;=\; \rho_\theta\Big(\tfrac{1}{5}\sum_{k=1}^{5} \phi_\theta\big(u^d_{x_{i,k}}\big)\Big). \tag{7}$$

Because summation is permutation?invariant, h_i^o treats offensive players as a set. The parameter set $\{\phi_\theta,\, \rho_\theta\}$ is shared between offense and defense.

Pros/Cons of Deep Sets:

- **Pros:**
  - Simple to implement and fast to train.
  - Guaranteed permutation invariance.
- **Cons:**
  - Only models *additive* interactions among players?cannot capture higher?order synergy (e.g., how a particular pairing of two offensive players interacts nonlinearly).
  - Less flexible than a full attention?based aggregator.

1.1.2 (B) Set Transformer Encoder

A *Set Transformer* lee2019set uses multihead self?attention over the five embeddings to capture interactions among them before pooling. Below we denote:

$$U_i^o \;=\; \Big[\, u^o_{o_{i,1}},\; u^o_{o_{i,2}},\; \ldots,\; u^o_{o_{i,5}}\Big]^\mathsf{T} \;\in\; \mathbb{R}^{5\times d}, \quad U_i^d = \Big[\, u^d_{x_{i,1}},\; u^d_{x_{i,2}},\; \ldots,\; u^d_{x_{i,5}}\Big]^\mathsf{T} \;\in\; \mathbb{R}^{5\times d}.$$

We then apply L layers of multihead self?attention to each set separately (parameters are denoted by θ). For the *offensive* set, let

$$H_i^{o,(0)} \;=\; U_i^o \in \mathbb{R}^{5\times d}.$$

At Transformer layer $\ell = 1, \ldots, L$, we compute:

$$Q_i^{o,(\ell)} = H_i^{o,(\ell-1)} W_Q^{(\ell)} \in \mathbb{R}^{5\times m}, \quad K_i^{o,(\ell)} = H_i^{o,(\ell-1)} W_K^{(\ell)} \in \mathbb{R}^{5\times m}, \tag{8}$$

$$V_i^{o,(\ell)} = H_i^{o,(\ell-1)} W_V^{(\ell)} \in \mathbb{R}^{5\times m}, \tag{9}$$

$$\text{Attention}_i^{o,(\ell)} = \text{softmax}\Big(\tfrac{Q_i^{o,(\ell)} (K_i^{o,(\ell)})^\intercal}{\sqrt{m}}\Big) V_i^{o,(\ell)} \in \mathbb{R}^{5\times m}, \tag{10}$$

$$H_i^{o,(\ell)} = \text{LayerNorm}\Big(H_i^{o,(\ell-1)} + \text{MultiHeadAttention}\big(H_i^{o,(\ell-1)}\big)\Big), \tag{11}$$

$$H_i^{o,(\ell)} = \text{LayerNorm}\Big(H_i^{o,(\ell)} + \text{FFN}\big(H_i^{o,(\ell)}\big)\Big), \tag{12}$$

where:

- $W_Q^{(\ell)}, W_K^{(\ell)}, W_V^{(\ell)} \in \mathbb{R}^{d\times m}$ project to head dimension m,
- MultiHeadAttention concatenates the h parallel heads (with $\sqrt{m}$?scaling), giving a residual?connected feature map in $\mathbb{R}^{5\times d}$,
- FFN$(\cdot)$ is the position?wise two?layer feedforward network $\text{ReLU}(\cdot\, W_1 + b_1)\, W_2 + b_2$,
- LayerNorm$(\cdot)$ is the usual transformer layer normalization.

Finally, after L such layers, we obtain $H_i^{o,(L)} \in \mathbb{R}^{5\times d}$. We then *pool* (by mean or by a learned `Pooling` token) to produce a single vector:

$$h_i^o = \text{Pool}\Big(H_i^{o,(L)}\Big) \in \mathbb{R}^d. \tag{13}$$

An identical Set Transformer (with independent parameters) encodes the defensive lineup to obtain h_i^d.

Attention Variants Between Offense and Defense. Optionally, rather than encoding offense and defense *completely* independently, one can allow cross?attention so that offensive embeddings attend to defensive embeddings (and vice versa) before final pooling. Suppose we have $H_i^{o,(L)} \in \mathbb{R}^{5\times d}$ and $H_i^{d,(L)} \in \mathbb{R}^{5\times d}$. We can form the concatenated matrix

$$H_i^{\mathsf{OD}} = \begin{bmatrix} H_i^{o,(L)} \\ H_i^{d,(L)} \end{bmatrix} \in \mathbb{R}^{10\times d}.$$

We then compute a layer of cross?attention in four possible blocks (with residual connections):

$$\text{(1) Offense-to-Offense Self}: \quad H_i^{o\leftarrow o} = \text{MHA}\big(H_i^{o,(L)}, H_i^{o,(L)}, H_i^{o,(L)}\big), \tag{14}$$

$$\text{(2) Defense-to-Defense Self}: \quad H_i^{d\leftarrow d} = \text{MHA}\big(H_i^{d,(L)}, H_i^{d,(L)}, H_i^{d,(L)}\big), \tag{15}$$

$$\text{(3) Offense-to-Defense Cross}: \quad H_i^{o\leftarrow d} = \text{MHA}\big(Q = H_i^{o,(L)}, K = H_i^{d,(L)}, V = H_i^{d,(L)}\big), \tag{16}$$

$$\text{(4) Defense-to-Offense Cross}: \quad H_i^{d\leftarrow o} = \text{MHA}\big(Q = H_i^{d,(L)}, K = H_i^{o,(L)}, V = H_i^{o,(L)}\big). \tag{17}$$

Here $\text{MHA}(Q, K, V)$ denotes the usual multihead attention block:

$$\text{MHA}(Q,K,V) = \text{Concat}\big(\text{head}_1, \ldots, \text{head}_h\big) W_O, \quad \text{head}_\ell = \text{softmax}\Big(\tfrac{Q W_Q^{(\ell)} (K W_K^{(\ell)})^\intercal}{\sqrt{m}}\Big)(V W_V^{(\ell)}),$$

with $W_Q^{(\ell)}, W_K^{(\ell)}, W_V^{(\ell)} \in \mathbb{R}^{d\times m}$ and $W_O \in \mathbb{R}^{(h\,m)\times d}$. Each block (1)?(4) is followed by residual skip and LayerNorm:

$$\widetilde{H}_i^{o\leftarrow o} = \text{LayerNorm}\big(H_i^{o,(L)} + H_i^{o\leftarrow o}\big), \quad \widetilde{H}_i^{o\leftarrow d} = \text{LayerNorm}\big(H_i^{o,(L)} + H_i^{o\leftarrow d}\big),$$

$$\widetilde{H}_i^{d\leftarrow d} = \text{LayerNorm}\big(H_i^{d,(L)} + H_i^{d\leftarrow d}\big), \quad \widetilde{H}_i^{d\leftarrow o} = \text{LayerNorm}\big(H_i^{d,(L)} + H_i^{d\leftarrow o}\big).$$

Finally, one can either:

- **Concatenate and Pool:**

$$H_i^{\text{final}} = \begin{bmatrix} \widetilde{H}_i^{o\leftarrow o} \\ \widetilde{H}_i^{o\leftarrow d} \\ \widetilde{H}_i^{d\leftarrow o} \\ \widetilde{H}_i^{d\leftarrow d} \end{bmatrix} \in \mathbb{R}^{20\times d}, \quad h_i^{o,d} = \text{Pool}\big(H_i^{\text{final}}\big) \in \mathbb{R}^d,$$

  so that the final offensive+defensive summary merges all four interactions before pooling into a single d?dimensional vector.

- **Additive Fusion:**

$$h_i^o = \text{Pool}\big(\widetilde{H}_i^{o\leftarrow o} + \widetilde{H}_i^{o\leftarrow d}\big), \quad h_i^d = \text{Pool}\big(\widetilde{H}_i^{d\leftarrow d} + \widetilde{H}_i^{d\leftarrow o}\big).$$

In either case, we obtain h_i^o, $h_i^d \in \mathbb{R}^d$ capturing both intra?lineup (self) and cross?lineup attention patterns.

Final Prediction. Once h_i^o and h_i^d are obtained (Eqns. (**??**)), we concatenate with $\mathbf{g}_i$ to form

$$z_i = \big[h_i^o;\ h_i^d;\ \mathbf{g}_i\big] \in \mathbb{R}^{2d+p},$$

and pass through a prediction head (Eqn. (**??**)):

$$\hat{y}_i = \text{MLP}_\theta\big(z_i\big),$$

where MLP_θ typically consists of several fully?connected layers with ReLU activations and a final $\sigma(\cdot)$ if $\hat{y}_i$ is bounded or ordinal or a sigmoid if it is a probability.

Advantages / Considerations.

- **Permutation Invariance:** By using Deep Sets or a Set Transformer, the ordering of the five offensive (or defensive) players does not matter.
- **Intra?Lineup Interaction:** A Set Transformer can learn subtle interactions between specific pairs or triples of players (e.g., two offensive sharpshooters who work well together), beyond simple additive contributions.
- **Cross?Lineup Dependencies:** By adding cross?attention blocks between offense and defense, the model can learn ?match?up? effects directly (e.g., how a particular defender pair will collectively guard a pair of offensive players).
- **Interpretability:** These deep architectures are less interpretable than a linear RAPM model. However, one can still inspect attention weights to see which players or matchups the model focuses on.
- **Computational Cost:** Each possession now requires running a small transformer over 10 tokens (5 offense + 5 defense). If you also include cross?attention layers, the cost grows roughly as $O((10)^2\, d)$ per layer. In practice, 2?3 transformer layers suffice.

1.2 Other DeepRAPM Variants (Sketches)

Beyond DeepRAPM-Attn, one could consider:

1. **DeepRAPM-MLP:**
   - Simply embed each player j into u_j^o, $u_j^d \in \mathbb{R}^d$, sum them:

$$\tilde{h}_i^o = \sum_{j=1}^{5} u^o_{o_{i,j}}, \quad \tilde{h}_i^d = \sum_{k=1}^{5} u^d_{x_{i,k}},$$

- Form $z_i = [\,\tilde{h}_i^o;\ \tilde{h}_i^d;\ \mathbf{g}_i\,] \in \mathbb{R}^{2d+p}$ and pass through an MLP.

This is a deeper analog of classical additive RAPM: still permutation?invariant, but does not capture pairwise synergy explicitly.

2. **DeepRAPM-InteractionNet:**

   - Build a small ?interaction network? on the 10 players. Create a graph whose nodes are the 10 player tokens $\{\,o_{i,1}, \ldots, o_{i,5},\, x_{i,1}, \ldots, x_{i,5}\}$, and add edges whenever two players interact (e.g., all?to?all within offense and defense, plus cross edges).
   - Run a tiny Graph Neural Network (e.g., a 2?layer GCN) over this 10?node subgraph to produce final player features $\hat{u}_{\,o_{i,j}}$ and $\hat{u}_{\,x_{i,k}}$. Then sum or pool:

$$h_i^o = \tfrac{1}{5}\sum_{j=1}^{5} \hat{u}_{\,o_{i,j}}, \quad h_i^d = \tfrac{1}{5}\sum_{k=1}^{5} \hat{u}_{\,x_{i,k}}.$$

   - Concatenate $(h_i^o, h_i^d, \mathbf{g}_i)$ and feed to an MLP.

   This explicitly models a mini?graph of player interactions, capturing synergy or shared defensive coverage.

3. **DeepRAPM?RNN:**

   - If we wish to incorporate temporal sequences (e.g., multiple possessions in a row), we can feed the sequence of $(h_i^o, h_i^d, \mathbf{g}_i)$ pairs into an RNN (or GRU/LSTM) over possessions.
   - The hidden state $s_i = \mathrm{GRU}(s_{i-1},\, [h_i^o; h_i^d; \mathbf{g}_i])$ can capture momentum, fatigue, or other time?dependent effects.
   - The final prediction is $\hat{y}_i = \mathrm{MLP}(s_i)$ instead of Eqn. (**??**).

   This adds sequence awareness rather than treating each possession independently.

1.3 Counterfactual Lineup Predictions

One major advantage of learning continuous embeddings u_j^o, u_j^d per player is that we can predict counterfactual lineups. Suppose we want to estimate what would happen if we replaced one player on offense (say $o_{i,3}$) with a different player ℓ. We simply substitute u_ℓ^o in place of $u_{o_{i,3}}^o$ and re?compute:

$$h_i^{o,(\mathrm{new})} = \mathrm{SetEncoder}_\theta\Big(\{\,u_{o_{i,1}}^o,\ u_{o_{i,2}}^o,\ u_\ell^o,\ u_{o_{i,4}}^o,\ u_{o_{i,5}}^o\}\Big), \quad h_i^d \text{ unchanged.}$$

Then the predicted outcome becomes

$$\hat{y}_i^{(\mathrm{new})} \;=\; \mathrm{MLP}_\theta\Big(\big[h_i^{o,(\mathrm{new})};\ h_i^d;\ \mathbf{g}_i\big]\Big).$$

Because all parameters θ (including every u_j^o, u_j^d) are learned jointly across all training data, the model implicitly understands how each player?s latent embedding interacts with any subset of four other offensive players and the five defensive players. Thus, counterfactual insertion of player ℓ yields a coherent prediction.

1.4 Summary of DeepRAPM Frameworks

- We replace scalar offensive/defensive coefficients with **learned embeddings** $u_j^o,\, u_j^d \in \mathbb{R}^d$ for each player j.
- We aggregate a set of five embeddings into a lineup embedding via either:
  - DeepSets (additive pooling after an MLP), or

  - SetTransformer (multihead self?attention layers over 5 tokens).

- Optionally, we add *cross?attention* blocks so that offensive and defensive sets can interact before pooling.
- We fuse $(h_i^o,\ h_i^d,\ \mathbf{g}_i)$ via an MLP to produce $\hat{y}_i$.
- This generalizes classical RAPM by capturing intra?lineup synergy, lineup?versus?lineup matchups, and temporal dynamics (if we add an RNN on top).
- Counterfactual lineup predictions are trivial: replace one player?s embedding and recompute h^o or h^d.

This deep?learning framework thus extends RAPM from a linear additive model to a *permutation?invariant*, *attention?based* architecture capable of capturing rich interactions among players, defensive matchups, and gamestate features.